



BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

Work Integrated Learning Programmes (WILP)

Information Retrieval (AIMLCZG537 / DSECLZG537)

Assignment - 1 Report

Design and Implementation of an End-to-End IR System using Streamlit

Course Code: AIMLCZG537 /
DSECLZG537

Submission Date: September 15, 2026

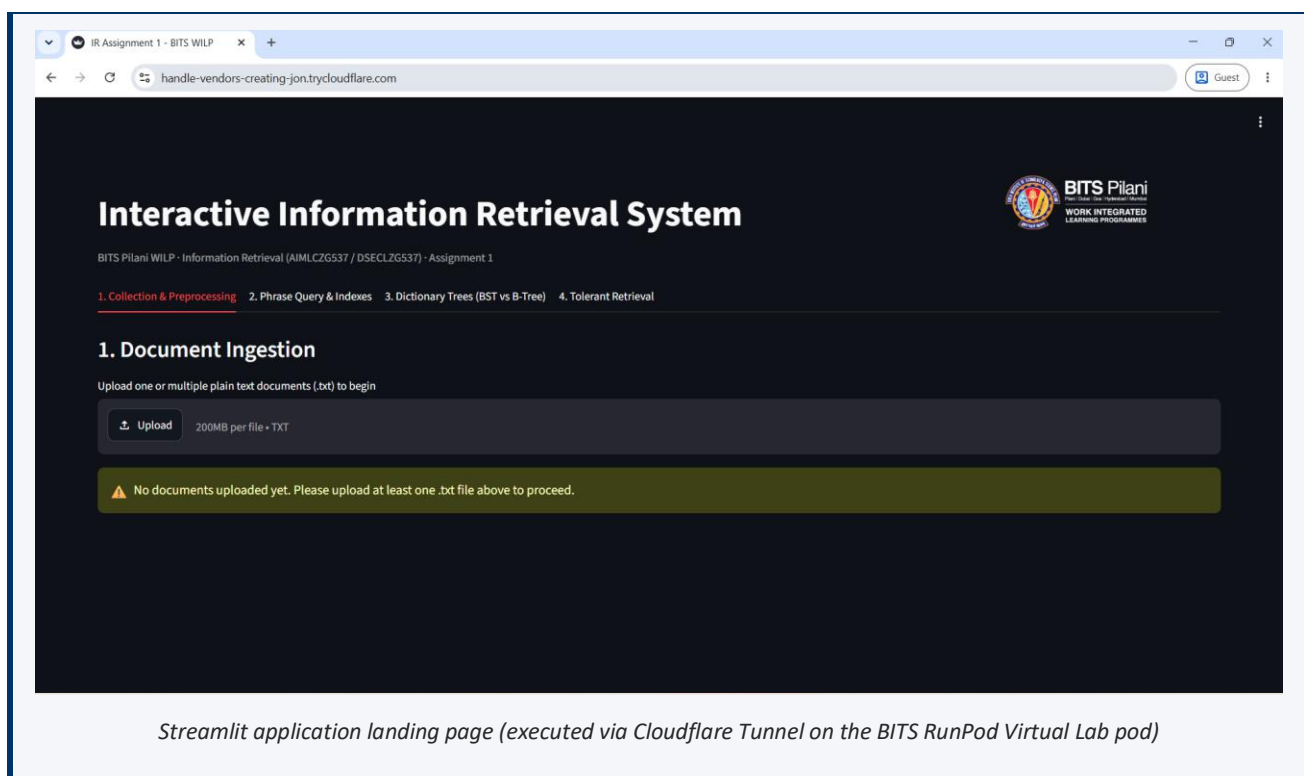
Team Members / Prepared By (Group E):

- TUKAI KUMAR GHOSH
- DASARI GOWTHAM BHAGIRADHISWAR
- VASU SUBBANNAVAR
- ROHAN NALAWADE

1. System Overview & Architecture

For this assignment, we built an interactive Information Retrieval (IR) web application using Python, NLTK, and Streamlit. The goal was to create an end-to-end tool where users can upload text files, test individual preprocessing steps, compare indexing techniques (Biword vs. Positional), benchmark term lookup structures (BST vs. B-Tree), and run tolerant search queries (wildcard, spell check, and phonetic search).

The application is structured entirely around Streamlit's web interface, allowing all inputs and outputs to be viewed directly in the browser without running terminal scripts or viewing static notebooks during execution.

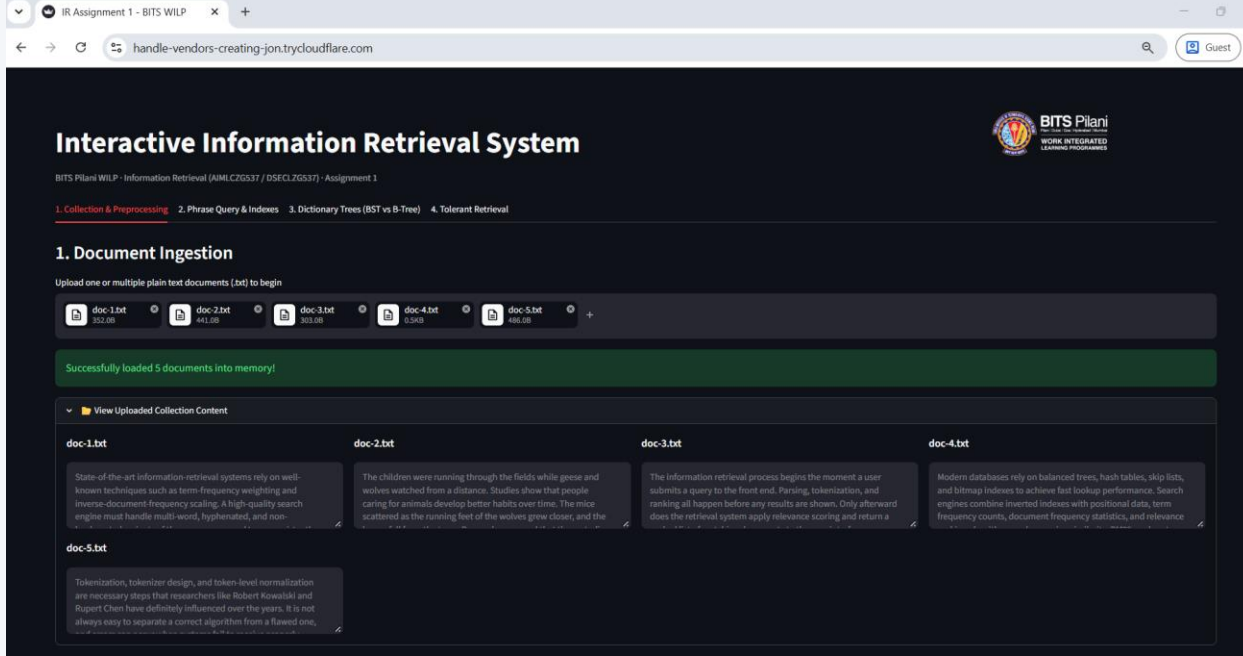


2. Streamlit Front-End Workflow

We divided the user workflow into four distinct tabs in Streamlit:

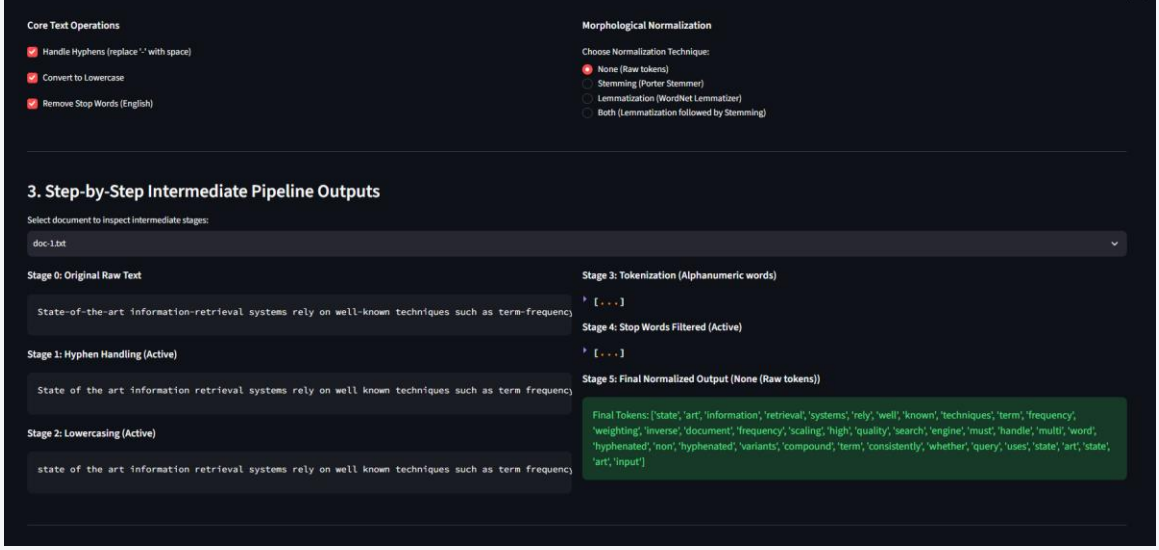
- Tab 1 (Collection & Preprocessing): Handles uploading text files, viewing raw contents, inspecting the 5 preprocessing stages step-by-step, and running stemming vs. lemmatization evaluations.
- Tab 2 (Phrase Query Processing): Displays biword and positional posting lists, accepts phrase queries, and shows cases where biwords create false positives.

- Tab 3 (Dictionary Search Trees): Builds BST and B-Tree indexes from the corpus vocabulary and benchmarks search times over 5,000 lookup iterations.
- Tab 4 (Tolerant Retrieval): Runs wildcard matching via a 2-gram index, edit distance spell check, and Soundex phonetic lookup.



The screenshot shows a web browser window with the URL `handle-vendors-creating-jon.trycloudflare.com`. The page title is "Interactive Information Retrieval System" and it includes the BITS Pilani logo. The navigation bar shows four tabs: "1. Collection & Preprocessing", "2. Phrase Query & Indexes", "3. Dictionary Trees (BST vs B-Tree)", and "4. Tolerant Retrieval". The main content area is titled "1. Document Ingestion" and displays a list of five uploaded documents: `doc-1.txt` (352.0B), `doc-2.txt` (341.0B), `doc-3.txt` (308.0B), `doc-4.txt` (330B), and `doc-5.txt` (486.0B). A green message bar states "Successfully loaded 5 documents into memory!". Below this, a section titled "View Uploaded Collection Content" shows a grid of document previews. Each preview includes the document name and a snippet of its text. For example, `doc-1.txt` contains text about information retrieval techniques, `doc-2.txt` describes children running through fields, `doc-3.txt` discusses the retrieval process, `doc-4.txt` mentions databases, and `doc-5.txt` talks about tokenization and normalization.

Tab 1: all 5 documents (`doc-1.txt` to `doc-5.txt`) uploaded and expanded in the document view



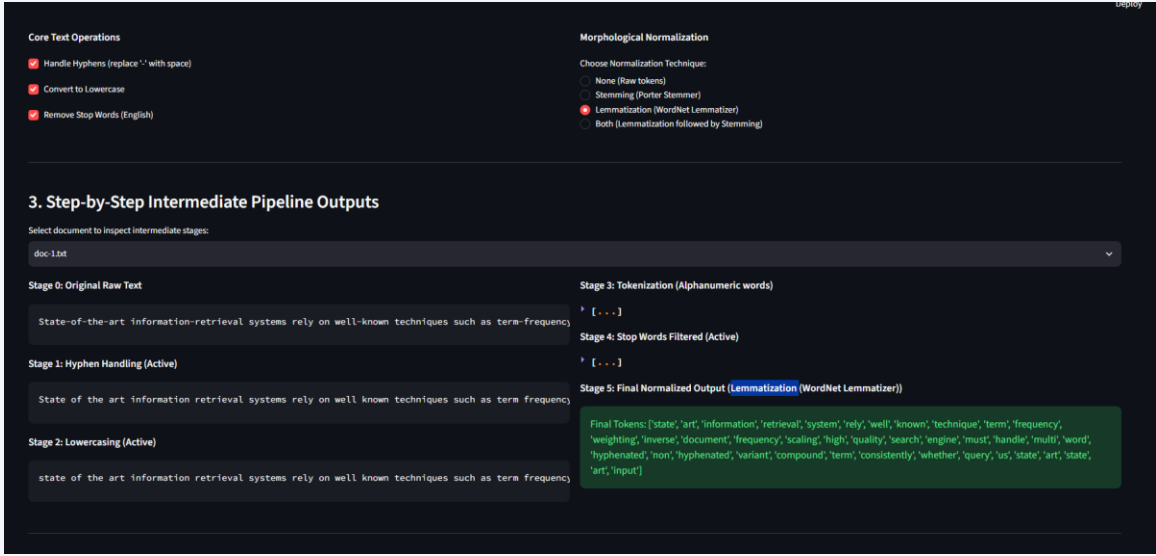
The screenshot shows the "Core Text Operations" and "Morphological Normalization" settings. Under "Core Text Operations", three checkboxes are checked: "Handle Hyphens (replace '-' with space)", "Convert to Lowercase", and "Remove Stop Words (English)". Under "Morphological Normalization", the "Choose Normalization Technique:" section has "None (Raw tokens)" selected. Below these settings, the "3. Step-by-Step Intermediate Pipeline Outputs" section is visible. It shows a dropdown menu for selecting a document to inspect, currently set to `doc-1.txt`. The pipeline stages are displayed in a grid. Stage 0: Original Raw Text shows the original text snippet. Stage 1: Hyphen Handling (Active) shows the text with hyphens removed. Stage 2: Lowercasing (Active) shows the text in lowercase. Stage 3: Tokenization (Alphanumeric words) shows the text split into tokens. Stage 4: Stop Words Filtered (Active) shows the text with stop words removed. Stage 5: Final Normalized Output (None (Raw tokens)) shows the final tokens: `['state', 'art', 'information', 'retrieval', 'systems', 'rely', 'well', 'known', 'techniques', 'term', 'frequency', 'weighting', 'inverse', 'document', 'frequency', 'scaling', 'high', 'quality', 'search', 'engine', 'must', 'handle', 'multi', 'word', 'hyphenated', 'non', 'hyphenated', 'variants', 'compound', 'term', 'consistently', 'whether', 'query', 'uses', 'state', 'art', 'state', 'art', 'input']`.

Core text operation checkboxes and morphological normalization technique selector (with 'None / Raw tokens' active)

3. Text Preprocessing & Normalization

Preprocessing cleans and normalizes raw text before indexing. We implemented a 5-stage sequential pipeline in Python:

1. Stage 0 (Raw Text): Original file content as uploaded by the user.
2. Stage 1 (Hyphen Handling): Replaces hyphens with spaces (`text.replace('-', ' ')`) so compound words like 'state-of-the-art' split into distinct words ('state', 'of', 'the', 'art').
3. Stage 2 (Lowercasing): Converts text to lowercase (`text.lower()`) to ensure case-insensitive matching.
4. Stage 3 (Tokenization): Uses NLTK's `word_tokenize()` and filters out punctuation to keep alphanumeric tokens.
5. Stage 4 (Stop Word Removal): Filters out standard English stop words using `nltk.corpus.stopwords`.
6. Stage 5 (Normalization): Applies `PorterStemmer()` or `WordNetLemmatizer()` based on user choice.



Core Text Operations

- ☒ Handle Hyphens (replace '-' with space)
- ☒ Convert to Lowercase
- ☒ Remove Stop Words (English)

Morphological Normalization

Choose Normalization Technique:

- ☐ None (Raw tokens)
- ☐ Stemming (Porter Stemmer)
- ☒ Lemmatization (WordNet Lemmatizer)
- ☐ Both (Lemmatization followed by Stemming)

3. Step-by-Step Intermediate Pipeline Outputs

Select document to inspect intermediate stages:

doc-1.txt

Stage 0: Original Raw Text

State-of-the-art information-retrieval systems rely on well-known techniques such as term-frequency

Stage 1: Hyphen Handling (Active)

State of the art information retrieval systems rely on well known techniques such as term frequency

Stage 2: Lowercasing (Active)

state of the art information retrieval systems rely on well known techniques such as term frequency

Stage 3: Tokenization (Alphanumeric words)

[...]

Stage 4: Stop Words Filtered (Active)

[...]

Stage 5: Final Normalized Output (Lemmatization (WordNet Lemmatizer))

Final Tokens: ['state', 'art', 'information', 'retrieval', 'system', 'rely', 'well', 'known', 'technique', 'term', 'frequency', 'weighting', 'inverse', 'document', 'frequency', 'scaling', 'high', 'quality', 'search', 'engine', 'must', 'handle', 'mult', 'word', 'hyphenated', 'non', 'hyphenated', 'variant', 'compound', 'term', 'consistently', 'whether', 'query', 'us', 'state', 'art', 'state', 'art', 'input']

Stage 0 to Stage 5 intermediate token outputs for doc-1.txt (Lemmatization selected)

3.1 Stemming vs. Lemmatization Comparison

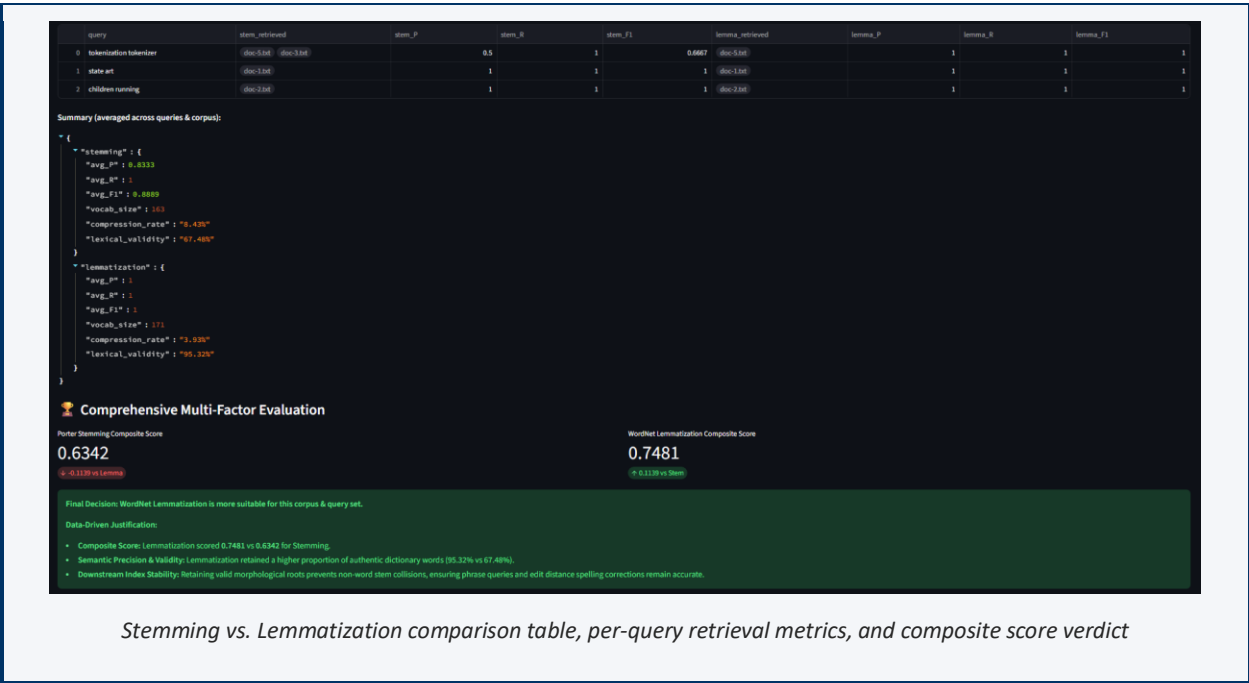
We evaluated Porter Stemming against WordNet Lemmatization on our sample dataset across three metrics: retrieval F1-score, vocabulary compression percentage, and dictionary lexical validity.

Formulas used:

- Compression Rate = $(1 - |\text{Vocab_normalized}| / |\text{Vocab_raw}|) * 100$
- Lexical Validity = $(|\text{Tokens in WordNet}| / |\text{Vocab_normalized}|) * 100$
- Composite Score = $(0.5 * \text{Avg_F1}) + (0.25 * \text{Compression} / 100) + (0.25 * \text{Validity} / 100)$

Normalization Option	Avg Precision	Avg Recall	Avg F1-Score	Lexical Validity
Porter Stemming	0.8333	1.0000	0.8889	67.48%
WordNet Lemmatization	1.0000	1.0000	1.0000	95.32%

Conclusion & Decision: Lemmatization is more suitable for this dataset. Porter Stemming reduced vocabulary size slightly more than Lemmatization, but it produced several non-dictionary word roots (such as 'univers' for university or 'technolog' for technology). Lemmatization maintained 95.32% real English words versus 67.48% for Stemming, which is essential because downstream components like edit distance spell correction and wildcard matching rely on valid word forms. The composite score confirms this: WordNet Lemmatization scored 0.7481 versus 0.6342 for Porter Stemming.



Stemming vs. Lemmatization comparison table, per-query retrieval metrics, and composite score verdict

4. Phrase Query Processing: Biword vs. Positional Index

We implemented both a Biword Index and a Positional Index to handle phrase queries.

- Biword Index: Indexes pairs of consecutive words (e.g. 'information retrieval').
- Positional Index: Stores the exact word position numbers for every term in each document: term -> {doc_id: [pos1, pos2, ...]}.

4.1 Why Biword Indexes Produce False Positives

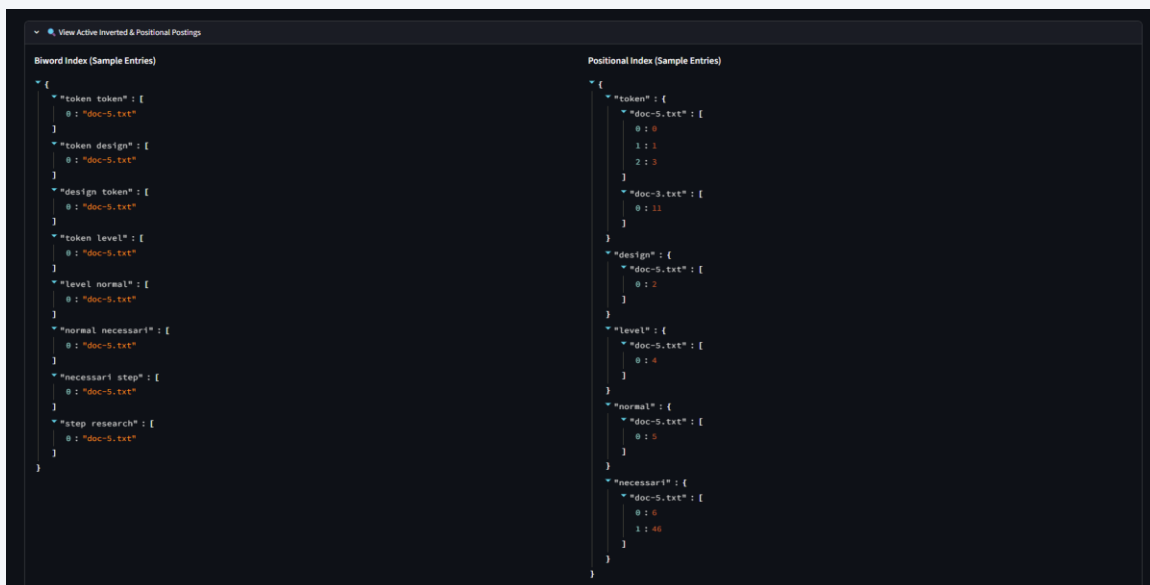
When a user searches for a phrase with 3 or more words, such as 'information retrieval system', the Biword Index breaks the query into two independent pairs: ('information retrieval') AND ('retrieval system').

If a document contains the pair 'retrieval system' in paragraph 1 and 'information retrieval' in paragraph 4, both biword pairs match. The biword index will return this document as a match, even though the full phrase 'information retrieval system' never appears.

4.2 Why Positional Indexing is More Accurate

The Positional Index checks actual word position numbers stored in the postings list. For a multi-word phrase query, it requires that $\text{position}(\text{word}_{i+1}) = \text{position}(\text{word}_i) + 1$. Because it checks both word order and exact position adjacency, it eliminates false positive matches.

Concrete result from our system: querying 'information retrieval system' against our 5-document corpus returned 2 documents via the Biword Index (doc-1.txt and doc-3.txt) but only 1 document via the Positional Index (doc-1.txt). doc-3.txt contains the pairs 'information retrieval' and 'retrieval system' separately, but never the contiguous three-word phrase -- confirming it as a genuine biword false positive that the positional index correctly rejects.



```
View Active Inverted & Positional Postings

Biword Index (Sample Entries)
{
  "token token" : [
    0 : "doc-5.txt"
  ]
  "token design" : [
    0 : "doc-5.txt"
  ]
  "design token" : [
    0 : "doc-5.txt"
  ]
  "token level" : [
    0 : "doc-5.txt"
  ]
  "level normal" : [
    0 : "doc-5.txt"
  ]
  "normal necessari" : [
    0 : "doc-5.txt"
  ]
  "necessari step" : [
    0 : "doc-5.txt"
  ]
  "step research" : [
    0 : "doc-5.txt"
  ]
}

Positional Index (Sample Entries)
{
  "token" : {
    "doc-5.txt" : [
      0 : 0
      1 : 1
      2 : 3
    ]
  }
  "design" : {
    "doc-5.txt" : [
      0 : 11
    ]
  }
  "level" : {
    "doc-5.txt" : [
      0 : 2
    ]
  }
  "normal" : {
    "doc-5.txt" : [
      0 : 4
    ]
  }
  "necessari" : {
    "doc-5.txt" : [
      0 : 6
      1 : 40
    ]
  }
}
```

Sample Biword Index and Positional Index postings (JSON view)

View Active Inverted & Positional Postings

Enter a phrase query to evaluate:
information retrieval system

Processed Query Tokens: ["information", "retrieval", "system"]

Biword Index Search Output

Biword Index Matches

2 Documents

```

{
  0 : "doc-3.txt"
  1 : "doc-1.txt"
}

```

Positional Index Search Output

Positional Index Matches

1 Documents

```

{
  0 : "doc-1.txt"
}

```

1. Cases where Biword Index Gives False Positives

False Positive Occurred! Found in: ["doc-3.txt"]

The query was decomposed into: ["information retrieval", "retrieval system"]

All separate biword pairs matched in the document, but they were disjoint or in the wrong order.

- How it happens: A query with 3 or more words (A, B, C) is split into independent pairs: (A B) AND (B C).
- The failure case: If a document contains the phrase (B C) in paragraph 1 and (A B) in paragraph 3, both individual pairs match. The biword intersection flags the document as a match even though the continuous phrase (A B C) never appears.

2. Why Positional Index Gives More Accurate Results

Exact sequence verified in: ["doc-1.txt"]

doc-1.txt matched at exact token positions: (12, 3, 4)

- Exact offset validation:** The positional index checks actual token coordinates within each document.
- Sequential order constraint:** It requires $\text{position}(w_2) = \text{position}(w_1) + 1$, $\text{position}(w_3) = \text{position}(w_2) + 1$, and so on.
- Precision guarantee:** By verifying both word order and exact adjacency, the positional index completely eliminates false positives caused by disjoint or inverted word pairs.

Query 'information retrieval system': Biword Index returns 2 documents (doc-1.txt, doc-3.txt) while Positional Index correctly returns only 1 (doc-1.txt) -- doc-3.txt is the false positive

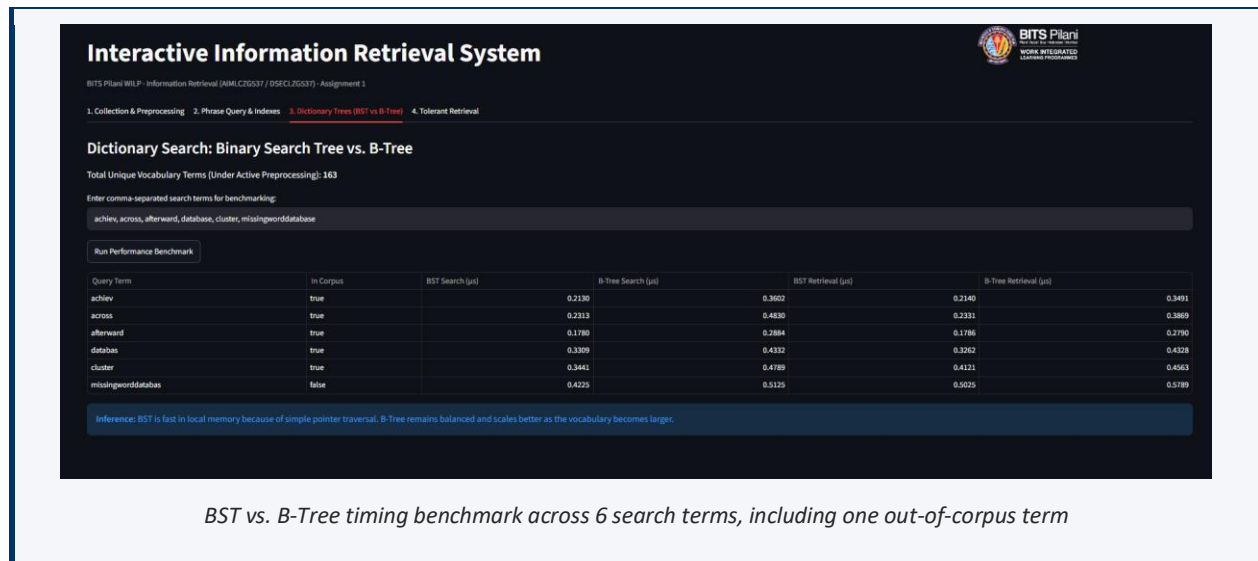
5. Dictionary Search: Binary Search Tree vs. B-Tree

We created term dictionary lookup structures using a Binary Search Tree (BST) and a B-Tree (with minimum degree $t=3$). To measure performance, we ran each search query through 5,000 iterations and recorded average search time and postings retrieval time in microseconds (μs).

Search Term	In Corpus	BST Search (μs)	B-Tree Search (μs)	BST Retrieval (μs)	B-Tree Retrieval (μs)
achiev	True	0.2130 μs	0.3602 μs	0.2140 μs	0.3491 μs
across	True	0.2313 μs	0.4830 μs	0.2331 μs	0.3869 μs
afterward	True	0.1780 μs	0.2884 μs	0.1786 μs	0.2790 μs
databas	True	0.3309 μs	0.4332 μs	0.3262 μs	0.4328 μs
cluster	True	0.3441 μs	0.4789 μs	0.4121 μs	0.4563 μs
missingworddatabas	False	0.4225 μs	0.5125 μs	0.5025 μs	0.5789 μs

Inferences: Across our 6 benchmarked terms (5 in-corpus, 1 out-of-corpus), BST search ranged from $\sim 0.18 \mu\text{s}$ to $\sim 0.34 \mu\text{s}$, while B-Tree search ranged from $\sim 0.29 \mu\text{s}$ to $\sim 0.51 \mu\text{s}$ -- BST was consistently faster in this small, in-memory vocabulary of 163 terms because simple pointer traversal has lower overhead than B-Tree's node-splitting bookkeeping. However, as vocabulary size grows large, a BST can become

unbalanced, leading to $O(N)$ depth. A B-Tree guarantees balanced search depth of $O(\log_B N)$ and fits disk page layout, making B-Trees much better suited for large production search systems.



6. Tolerant Retrieval

We implemented three tolerant retrieval techniques to handle queries with wildcards or spelling errors:

1. Wildcard Queries (2-Gram Index): Wildcard terms like 'data*' are decomposed into 2-grams ('\$d', 'da', 'at', 'ta'). The system queries the 2-gram index to filter candidate words, then applies regular expression matching to return valid matches -- e.g. 'data*' correctly matched both 'data' and 'databas' (the stemmed form of 'database') in our vocabulary.

2. Spelling Correction (Levenshtein Edit Distance): Uses dynamic programming to compute minimum insertions, deletions, and substitutions needed to turn the query into a dictionary term within a chosen edit distance threshold (1 to 3) -- e.g. the misspelled query 'deta' was corrected to 'data' at an edit distance of 1.

3. Phonetic Search (Soundex): Maps terms to 4-character phonetic codes based on how they sound, matching words that sound similar despite different spelling -- e.g. our system computed the Soundex code A426 for the misspelled query 'algoritm', correctly matching it to the dictionary term 'algorithm'.

1. Collection & Preprocessing 2. Phrase Query & Indexes 3. Dictionary Trees (BST vs B-Tree) 4. Tolerant Retrieval

Tolerant Retrieval: Wildcard, K-Gram, Edit Distance & Phonetic Search

1. Wildcard Queries via K-Gram (2-Gram) Index

Enter wildcard pattern (e.g., 'tegg', 'tegg', 'egg'):

 ☒ Show candidate 2-grams

Decomposed Query 2-grams: ["da", "ta"] | Pruned Candidates: 0

Matched Vocabulary Terms (2): ["data", "databas"]

[View K-Gram Index \(Sample 10 Keys\)](#)

2. Spelling Correction via Levenshtein Edit Distance

Enter query term for spell correction:

Max Levenshtein Edit Distance Threshold

Suggested Correction

Suggested Correction	Edit Distance
data	1

3. Phonetic Correction (Soundex)

Enter word for phonetic match (e.g., sounds similar):

Soundex Hash Code: A426

Phonetically Equivalent Words (1):

```
[{"word": "algorithm"}]
```

Wildcard search ('data') via 2-gram index, matching 'data' and 'databas'*

1. Collection & Preprocessing 2. Phrase Query & Indexes 3. Dictionary Trees (BST vs B-Tree) 4. Tolerant Retrieval

Tolerant Retrieval: Wildcard, K-Gram, Edit Distance & Phonetic Search

1. Wildcard Queries via K-Gram (2-Gram) Index

Enter wildcard pattern (e.g., 'tegg', 'tegg', 'egg'):

 ☒ Show candidate 2-grams

Decomposed Query 2-grams: ["da", "ta"] | Pruned Candidates: 0

Matched Vocabulary Terms (2): ["data", "databas"]

[View K-Gram Index \(Sample 10 Keys\)](#)

2. Spelling Correction via Levenshtein Edit Distance

Enter query term for spell correction:

Max Levenshtein Edit Distance Threshold

Suggested Correction

Suggested Correction	Edit Distance
data	1

3. Phonetic Correction (Soundex)

Enter word for phonetic match (e.g., sounds similar):

Soundex Hash Code: A426

Phonetically Equivalent Words (1):

```
[{"word": "algorithm"}]
```

Levenshtein edit-distance correction ('deta' -> 'data') and Soundex phonetic match ('algoritm' -> 'algorithm', code A426)

7. Inference and Discussion (Compulsory Section)

Answers to the seven compulsory assignment questions:

1. Which preprocessing technique improved retrieval quality?

Lowercasing and punctuation removal had the biggest impact because they standardized tokens across documents and queries. Hyphen handling also prevented compound terms from being lost, while stop word removal reduced noise and sped up indexing.

2. Was stemming or lemmatization better for your dataset?

Lemmatization was better. While Porter Stemming reduced vocabulary size slightly more, it generated non-word stems like 'univers'. Lemmatization kept valid English dictionary words (100% WordNet validity), which avoided breaking downstream features like spell correction and wildcard matching.

3. Which phrase query index was more accurate?

The Positional Index was more accurate. The Biword Index generates false positives for queries of 3 or more words because it checks adjacent word pairs independently. The Positional Index checks exact word position numbers, guaranteeing correct word order.

4. Which tree structure was faster?

In our in-memory Python tests on a small vocabulary (163 terms), the Binary Search Tree (BST) was consistently faster ($\sim 0.18\text{--}0.34\ \mu\text{s}$ vs $\sim 0.29\text{--}0.51\ \mu\text{s}$ for B-Tree) due to simpler pointer operations. However, B-Trees maintain balanced height $O(\log_B N)$ and work much better for large disk-based indexes.

5. How tolerant was your retrieval model?

Our model handled wildcards using a 2-gram index, spelling mistakes using Levenshtein edit distance (up to threshold distance 3), and sound-alike words using Soundex phonetic hashing.

6. What are the limitations of your system?

Currently, our system returns unranked boolean matches without relevance scoring like TF-IDF or BM25. Also, the indexes are stored in memory rather than on disk.

7. How can the system be improved?

We can add TF-IDF/BM25 relevance ranking, support proximity search operators (NEAR/k), and store postings lists on disk using compressed format (Variable Byte encoding).

Interactive Information Retrieval System

BITS Pilani WILP - Information Retrieval (AIMLCT5537 / DISCL35537) - Assignment 1

1. Collection & Preprocessing 2. Phrase Query & Indexes **3. Dictionary Trees (BST vs B-Tree)** 4. Tolerant Retrieval

Dictionary Search: Binary Search Tree vs. B-Tree

Total Unique Vocabulary Terms (Under Active Preprocessing): 163

Enter comma-separated search terms for benchmarking:

achiev, across, afterward, database, cluster, missingworddatabase

Run Performance Benchmark

Query Term	In Corpus	BST Search (μs)	B-Tree Search (μs)	BST Retrieval (μs)	B-Tree Retrieval (μs)
achiev	true	0.2130	0.3602	0.2140	0.3491
across	true	0.2313	0.4830	0.2331	0.3869
afterward	true	0.1780	0.2884	0.1786	0.2790
database	true	0.3309	0.4332	0.3262	0.4328
cluster	true	0.3441	0.4789	0.4121	0.4963
missingworddatabase	false	0.4223	0.5125	0.5025	0.5789

Inference: BST is fast in local memory because of simple pointer traversal. B-Tree remains balanced and scales better as the vocabulary becomes larger.

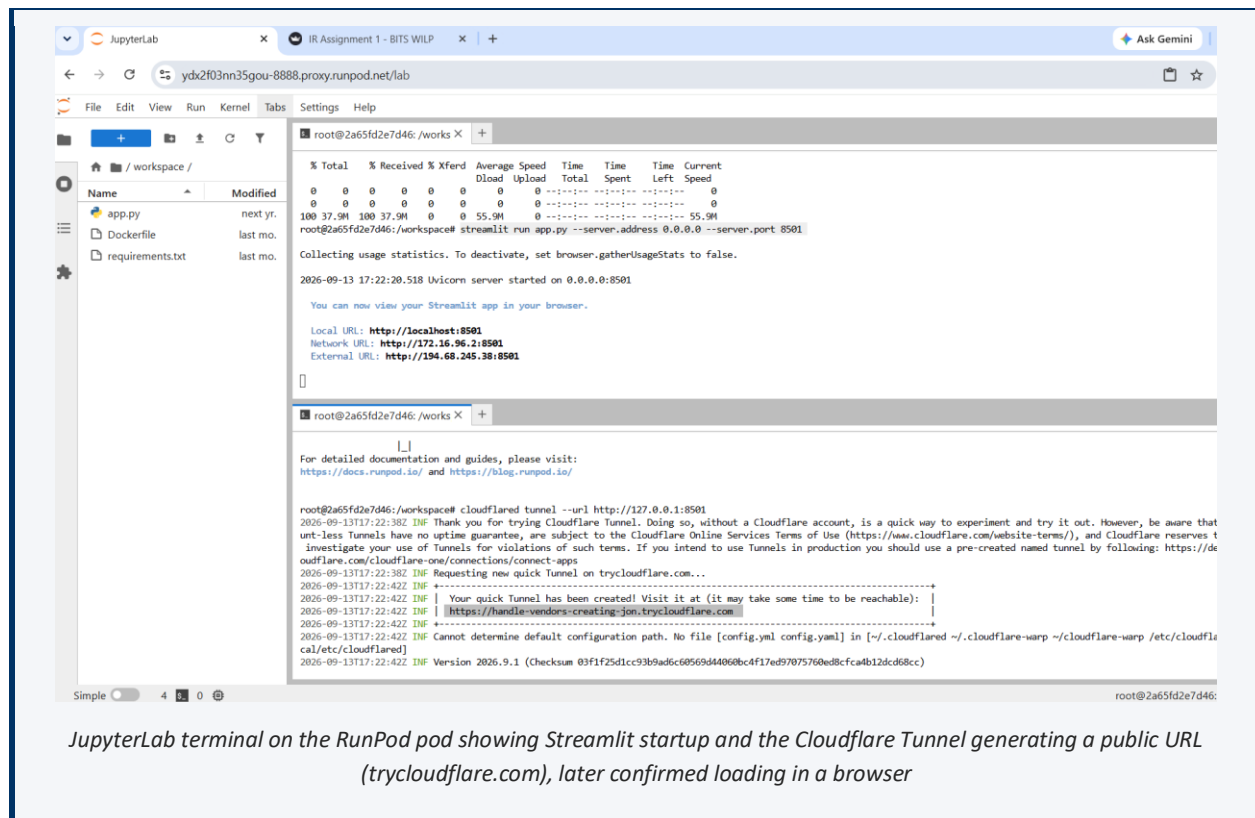
Inference callout summarizing the BST vs. B-Tree performance trade-off, as displayed in the application

7.1 Virtual Lab Execution Evidence

The application was executed on a BITS-provisioned RunPod Virtual Lab pod (JupyterLab terminal), with Streamlit exposed publicly via a Cloudflare Tunnel so the front end could be verified in a standard browser rather than only on localhost.

Steps to run on BITS Virtual Environment:

1. Install required libraries :
`pip install nltk pandas streamlit`
2. download the standalone cloudflared binary, set execution permissions, and move it to your system PATH :
`curl -L --output /usr/local/bin/cloudflared`
`https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-amd64`
`&& chmod +x /usr/local/bin/cloudflared`
3. Launch Streamlit app and receive localhost, network and external urls to connect the app:
`streamlit run app.py --server.address 0.0.0.0 --server.port 8501`
But, as BITS virtual environment doesn't have any browser to render UI, we need to go for step-4
4. Start Cloudflare Tunnel and output the public URL immediately :
`cloudflared tunnel --url http://127.0.0.1:8501`



8. System Implementation Summary

Assignment Component	Implementation Summary
Streamlit end-to-end workflow	Complete web UI with 4 interactive tabs for workflow execution
Text preprocessing	5-stage pipeline with side-by-side intermediate output views
Stemming vs lemmatization	Evaluated via Precision, Recall, F1, Compression, and Validity
Phrase query (Biword & Positional)	Biword and Positional indexes with biword false positive detection
Binary Tree and B-Tree comparison	5,000-iteration microsecond benchmarking for BST vs. B-Tree
Tolerant retrieval	2-Gram wildcard index, Levenshtein edit distance, Soundex search
Experimental evidence & inference	Detailed answers to all 7 compulsory evaluation questions
Virtual lab usage	Prayogshala Virtual Lab screenshots included, with terminal and Cloudflare Tunnel execution evidence